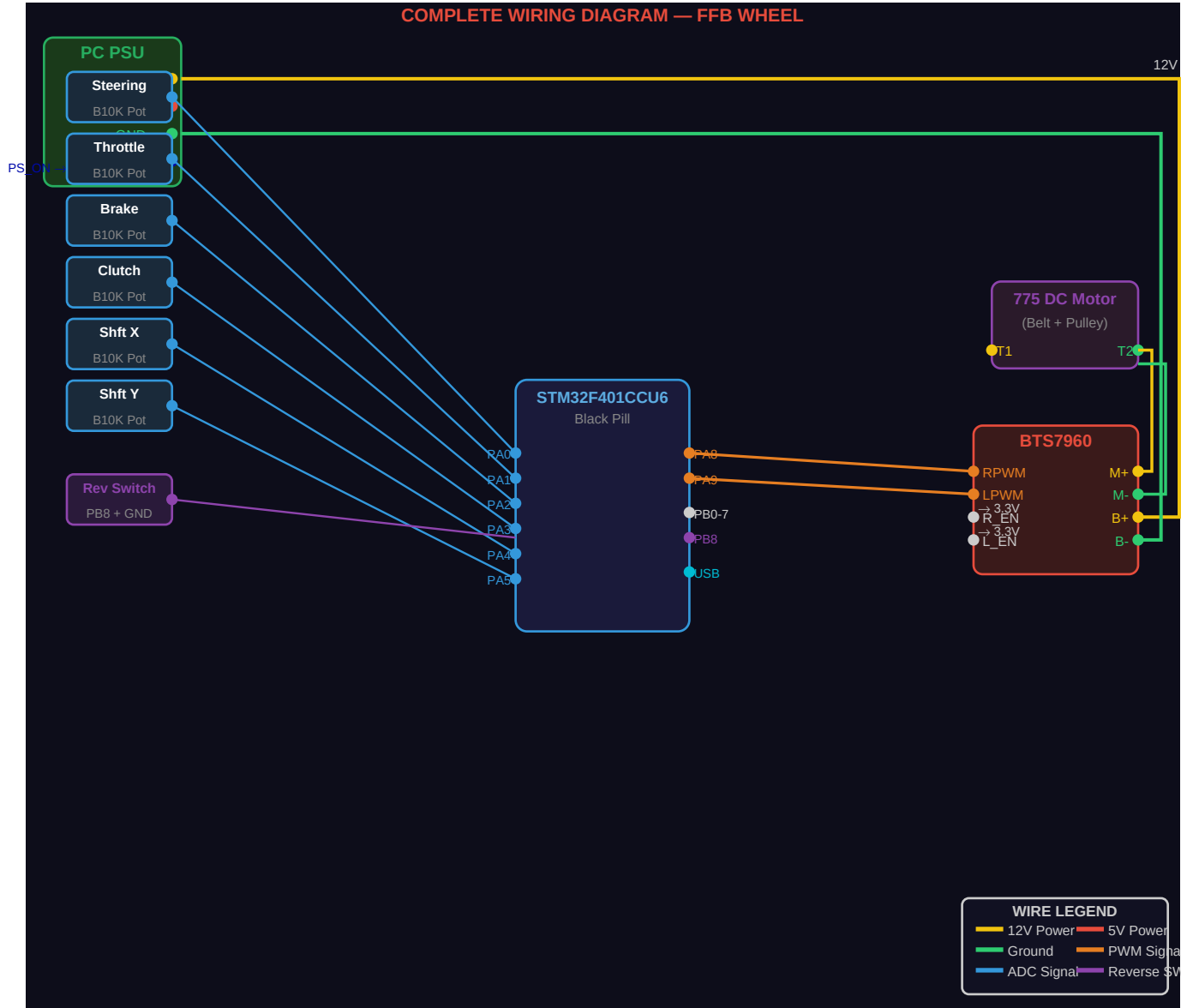


DIY FFB Racing Wheel

Complete Setup Guide — All Pending Steps + Wiring

STM32F401CCU6 • BTS7960 • 775 Motor • B10K Pot • 6-Speed + Reverse

1. Complete Wiring Diagram



BTS7960 Motor Driver Connections

BTS7960 Pin	Connect To
M+	775 Motor terminal 1
M-	775 Motor terminal 2

B+	12V PSU yellow wire — add 15A fuse on this line
B-	GND common
RPWM	PA8 on STM32 (TIM1 CH1 — forward PWM)
LPWM	PA9 on STM32 (TIM1 CH2 — reverse PWM)
R_EN	3.3V — tie HIGH permanently, no GPIO needed
L_EN	3.3V — tie HIGH permanently, no GPIO needed
VCC	5V from PSU red wire
GND	GND common

Potentiometer Connections (all 6 pots identical wiring)

Left pin	3.3V
Middle pin	ADC pin → PA0 Steering PA1 Throttle PA2 Brake PA3 Clutch PA4 Shifter X PA5 Shifter Y
Right pin	GND

Reverse Switch — PB8 Microswitch (NEW)

Wire a small microswitch under your shifter knob. One terminal to PB8, other terminal to GND. No resistor needed — firmware uses internal pull-up. Push knob DOWN while moving to 1st gear position = Reverse. Without pushing = normal 1st gear.

Microswitch terminal 1	PB8 on STM32
Microswitch terminal 2	GND common
No resistor needed	Firmware INPUT_PULLUP handles it

Pin Summary

STM32 Pin	Function
PA0	Steering pot ADC
PA1	Throttle ADC
PA2	Brake ADC
PA3	Clutch ADC
PA4	Shifter X ADC
PA5	Shifter Y ADC
PA8	BTS7960 RPWM — TIM1 CH1 (forward)
PA9	BTS7960 LPWM — TIM1 CH2 (reverse)
PA10	Optional BTS7960 R_EN (or tie to 3.3V)
PA11	USB D- — DO NOT USE FOR ANYTHING ELSE
PA12	USB D+ — DO NOT USE FOR ANYTHING ELSE
PB0–PB7	Wheel buttons 1–8 (INPUT_PULLUP, connect to GND)
PB8	Reverse gear microswitch (INPUT_PULLUP, connect to GND)

2. STM32CubeIDE Project Setup

Step 1 — Create Project

1	Open STM32CubeIDE
2	File → New → STM32 Project
3	In board selector type STM32F401CCU6 → select → Next
4	Name the project FFBWheel → Finish

Step 2 — Configure .ioc File (Pinout & Configuration)

Peripheral	Setting
USB_OTG_FS	Connectivity → USB_OTG_FS → Mode: Device Only
USB_DEVICE	Middleware → USB_DEVICE → Class: Custom HID
CDC (Serial)	Add second class for CDC — needed for config app COM port. Search GitHub for 'STM32 USB Composite HID CDC F401' template
ADC1	Enable ADC1 → Channels IN0,IN1,IN2,IN3,IN4,IN5 → Continuous Conversion → DMA: DMA2 Stream0, Circular mode
TIM1	Enable TIM1 → CH1: PWM Generation CH1 (PA8) → CH2: PWM Generation CH2 (PA9) → Prescaler: 3 → ARR: 999
TIM2	Enable TIM2 base → Prescaler: 83 → ARR: 999 → Enable Update Interrupt → This drives the 1kHz FFB loop
Clock	Set HCLK to 84 MHz → USB PLL48CLK auto-configured

Step 3 — Copy Source Files

After CubeMX generates code, copy these files into your project:

Core/Inc/	ffb_wheel.h, motor.h, adc_input.h, ffb_engine.h, config.h, serial_cmd.h, usb_hid_desc.h, ffb_timer.h, failsafe.h
Core/Src/	main.c, motor.c, adc_input.c, ffb_engine.c, config.c, serial_cmd.c, usb_hid_desc.c, ffb_timer.c, failsafe.c

In STM32CubeIDE: right-click Core/Inc → Refresh, right-click Core/Src → Refresh

3. USB Callback Hookups (Step 4 from README)

CubeMX generates USB driver files. You must manually add three small edits to hook the firmware into the USB data flow. These are the only manual edits required in generated files — everything else is in the provided source files.

Edit 1 — USB_DEVICE/App/usbd_hid_if.c

Find the CUSTOM_HID_OutEvent_FS function and replace it entirely:

```
extern void USBD_HID_OutCallback(uint8_t report_id, uint8_t *buf, uint16_t len); static int8_t CUSTOM_HID_OutEvent_FS(uint8_t event_idx, uint8_t state) { USBD_CUSTOM_HID_HandleTypeDef *hhid = (USBD_CUSTOM_HID_HandleTypeDef*)hUsbDeviceFS.pClassData; if (hhid != NULL) { uint8_t *buf = hhid->Report_buf; uint8_t rid = buf[0]; // first byte = report ID USBD_HID_OutCallback(rid, buf, CUSTOM_HID_EP0UT_SIZE); } return USBD_OK; }
```

Edit 2 — USB_DEVICE/App/usbd_cdc_if.c

Find the CDC_Receive_FS function and replace it:

```
extern void CDC_ReceiveCallback(uint8_t *buf, uint32_t len); static int8_t CDC_Receive_FS(uint8_t* Buf, uint32_t *Len) { CDC_ReceiveCallback(Buf, *Len); USBD_CDC_SetRxBuffer(&hUsbDeviceFS, &Buf[0]); USBD_CDC_ReceivePacket(&hUsbDeviceFS); return USBD_OK; }
```

Edit 3 — USB_DEVICE/App/usbd_desc.c (or usbd_hid_if.c)

Point the HID descriptor to our custom FFB descriptor:

```
#include "usb_hid_desc.h" // Find the descriptor table entry that references the HID report // descriptor and change it to: // .pReportDesc = HID_ReportDescriptor, // .ReportDescSize = HID_ReportDescriptorSize, // Exact location depends on your CubeMX version. // Search for "HID_MOUSE_ReportDesc" or "CUSTOM_HID_ReportDesc" // in the generated files and replace that reference.
```

Edit 4 — Core/Src/stm32f4xx_it.c

Add the TIM2 interrupt handler for the 1kHz FFB loop:

```
extern TIM_HandleTypeDef htim2; extern void FFB_Timer_Tick(void); void TIM2_IRQHandler(void) { if (__HAL_TIM_GET_FLAG(&htim2, TIM_FLAG_UPDATE) && __HAL_TIM_GET_IT_SOURCE(&htim2, TIM_IT_UPDATE)) { __HAL_TIM_CLEAR_IT(&htim2, TIM_IT_UPDATE); FFB_Timer_Tick(); // runs FFB calc + Motor_Set at exactly 1kHz } }
```

4. Build, Flash and First-Time Calibration

Step 5 — Build and Flash

1	Click Build button (hammer icon) in STM32CubeIDE
2	Fix any errors. Warnings about unused variables are OK
3	To enter DFU flash mode: hold BOOT0 button on Black Pill, then plug USB
4	Flash via: Run → Run in STM32CubeIDE OR STM32CubeProgrammer in DFU mode
5	After flash, unplug and replug normally (no BOOT0 held)
6	Check Windows Device Manager — should see 'FFB Wheel' gamepad + COM port

Step 6 — Config App + Calibration Order

Open index.html in Google Chrome. Click Connect and select the wheel COM port. Follow this exact calibration order on first setup:

#	Tab	Action
1	Wheel Setup tab	Set Rotation Range: 900, Pulley Ratio: 4.5, Pot Sweep: 200
2	Wheel Setup tab	Turn full LEFT lock → Set Left Lock. Center → Set Center. Full RIGHT → Set Right Lock
3	Pedals tab	Throttle: release → Set Released, press to floor → Set Full Press. Repeat for Brake and Clutch
4	Shifter tab	Move shifter to Left gate → Set Left Gate. Right gate → Set Right Gate. Forward → Set Forward. Reverse → Set Reverse
5	Shifter tab	Test reverse: push knob down + move to 1st gear position — gear display should show R in purple
6	FFB Tuning tab	Start: Spring 65%, Strength 70%, Max Torque 50%. Open a game and test feel
7	FFB Tuning tab	Increase torque slowly once confirmed working. Tune spring/damper to taste
8	Save bar	Click Save to Flash — all settings stored permanently

5. Failsafe & Timing System

Three automatic safety systems run in the background. They are already implemented in `failsafe.c` and `ffb_timer.c` — no configuration needed.

System	Description
IWDG Watchdog	Hardware watchdog timer. If the firmware freezes or crashes for any reason, the STM32 automatically resets after 500ms. On reset the motor PWM starts at 0. Pet call is inside <code>FFB_Timer_Tick</code> so it runs every 1ms automatically.
USB Timeout	If the USB cable is unplugged mid-game or the game closes while the motor is running, the motor stops within 200ms. Triggered when no FFB HID OUT report is received within the timeout window.
Stuck Motor Detection	If motor is commanded above 80% PWM for more than 2 seconds continuously but the steering pot shows less than 2 degrees of movement (belt snapped, pot unplugged, or mechanical jam), motor is cut immediately and a failsafe flag is sent to the web app.
Soft Lock	Motor torque is smoothly ramped to zero as the wheel approaches the configured lock-to-lock limit (within the last 8 degrees). Prevents hard slamming at limits.
1kHz Hardware Timer	The FFB calculation and <code>Motor_Set()</code> run from TIM2 interrupt at exactly 1kHz, not from <code>HAL_GetTick()</code> polling. This gives stable, jitter-free FFB timing even when USB interrupts are busy.

6. Troubleshooting

Problem	Solution
Wheel not in Device Manager	USB cable must be data cable (not charge-only). Check Black Pill is not still in DFU mode. Try different USB port.
COM port missing	CDC composite not set up correctly in CubeMX. Make sure both HID and CDC class handlers are registered. See README note on Composite HID+CDC.
FFB not working in game	Open <code>joy.cpl</code> (Windows Game Controllers) — test axes first. Enable FFB in game settings under steering wheel options. Run <code>dxdiag</code> to verify FFB device listed.
Motor runs on power-on	Steering pot center calibration is wrong. Re-do Set Center with wheel pointing straight.
Steering direction wrong	Use Invert toggle in Wheel Setup tab. Or physically swap M+ and M- wires on BTS7960.
Reverse not detected	Check microswitch wired to PB8 and GND. Check PB8 GPIO init in <code>adc_input.c</code> (GPIOB clock must be enabled). Test by shorting PB8 to GND directly.
Web app cannot connect	Must use Google Chrome v89+. Firefox does not support Web Serial API. Allow port permission popup when Chrome asks.
Failsafe triggered — stuck motor	Check belt is intact and pot wires are secure. Lower Max Torque limit. Increase stuck detection time threshold in <code>failsafe.c</code> if false-triggering.
FFB feels weak	Increase Spring and Strength sliders in FFB tab. Check Max Torque is not too low. Verify motor is getting 12V from PSU.
Gears not detecting correctly	Re-calibrate shifter gate thresholds. Move shifter fully into each gate before clicking Set. Check X/Y pot wiring (left pin = 3.3V, right = GND).

7. Safety Checklist

The 775 motor at 12V produces serious torque. Read every item below before powering on for the first time.

✓	Set Max Torque to 50% for first test. Increase slowly after confirming everything works.
✓	Add a physical E-STOP button: wire a switch between PS_ON (green wire on ATX PSU) and GND. Press = PSU off = motor dead.
✓	Fuse the 12V motor line: add a 15A blade fuse between PSU 12V and BTS7960 B+. BTS7960 can pass unlimited current without it.
✓	Add heatsink to BTS7960 if running sessions longer than 20 minutes. It gets hot under sustained FFB.
✓	Keep hands on the wheel during first test — do not let it spin freely at high torque.
✓	Test at low torque with someone who can pull the PSU cable if something goes wrong.
✓	Do not run motor wires alongside ADC/signal wires — motor switching noise will cause pot jitter and unstable FFB.
✓	Verify all GND connections share a common point. Floating grounds cause random motor behavior.

8. Gear System — 6-Speed + Reverse

Physical Layout (viewed from above)

Gate	Left (X low)	Center (X mid)	Right (X high)
Forward (Y high)	1st gear	3rd gear	5th gear
Neutral	.	Neutral	.
Reverse (Y low)	2nd gear	4th gear	6th gear

Reverse Gear Logic

Reverse is triggered when the PB8 microswitch is pressed (LOW) AND the shifter is in the 1st gear position (left gate, forward throw). Without PB8 pressed, the same position returns 1st gear as normal. This matches real H-pattern shifters where you must push down to engage reverse.

Shifter at 1st position, PB8 not pressed	→ GEAR_1 (1st gear)
Shifter at 1st position, PB8 pressed DOWN	→ GEAR_R (Reverse) — shown purple in web app
Any other position regardless of PB8	→ Normal gear 2–6 or Neutral

Custom FFB Wheel Firmware — STM32F401CCU6. All code is open and free. Web config app runs in Google Chrome with no installation required. Firmware built with STM32CubeIDE (free). No paid tools needed.